

2. Fundamento teórico

2.1. Documentación de requisitos

2.1.1. Concepto y propósito de la documentación de requisitos

La documentación de requisitos constituye uno de los pilares fundamentales de la Ingeniería de Requisitos, ya que proporciona un medio estructurado para capturar, organizar, comunicar y mantener las necesidades y restricciones identificadas durante el desarrollo de un sistema. Su propósito trasciende la simple recopilación de requisitos, al convertirse en el principal mecanismo de comunicación entre clientes, usuarios, analistas, desarrolladores, verificadores y demás partes interesadas durante todo el ciclo de vida del sistema. La norma IEEE/ISO/IEC 29148:2018 establece los procesos, productos de información y contenidos mínimos asociados a la ingeniería de requisitos, complementando los procesos definidos por ISO/IEC/IEEE 15288 para sistemas y por ISO/IEC/IEEE 12207 para software, garantizando así la coherencia entre la especificación de requisitos y el resto de los procesos de ingeniería del ciclo de vida [1], [2], [3].

Desde una perspectiva metodológica, la documentación de requisitos debe representar de forma precisa el problema que se desea resolver, evitando introducir decisiones de diseño prematuras. Asimismo, debe facilitar la trazabilidad entre las necesidades del negocio, los requisitos de sistema, los requisitos de software, los modelos de análisis y las actividades posteriores de implementación, verificación y validación. En este contexto, SWEBOK reconoce que una documentación de calidad constituye un factor determinante para reducir errores de interpretación y minimizar los costos derivados de cambios tardíos durante el desarrollo del software [4], [5].

2.1.2. Tipos de documentos de requisitos según IEEE/ISO/IEC 29148

La norma IEEE/ISO/IEC 29148:2018 define diversos productos de información destinados a documentar requisitos en diferentes niveles de abstracción. Cada documento responde a una audiencia específica y cumple un propósito particular dentro del proceso de ingeniería de requisitos, permitiendo mantener la separación entre necesidades de los interesados, requisitos del sistema y requisitos del software [1].

Cuadro 1: Principales documentos de requisitos definidos por IEEE/ISO/IEC 29148:2018

Documento	Propósito	Audiencia principal	Contenido predominante
StRS (Stakeholder Requirements Specification)	Documentar las necesidades, objetivos, restricciones y expectativas de los interesados.	Cliente, patrocinadores y usuarios finales.	Necesidades del negocio, objetivos estratégicos, restricciones y criterios de aceptación.
SyRS (System Requirements Specification)	Especificar los requisitos del sistema derivados de las necesidades de los interesados.	Arquitectos de sistemas e ingenieros de sistemas.	Funciones del sistema, interfaces, restricciones, desempeño y requisitos de calidad.
SRS (Software Requirements Specification)	Definir detalladamente los requisitos del software que implementará el sistema.	Analistas, desarrolladores, verificadores y responsables de pruebas.	Requisitos funcionales, no funcionales, interfaces, modelos de datos y restricciones técnicas.
OpsCon (Operational Concept)	Describir el contexto operacional del sistema y la interacción con usuarios y sistemas externos.	Usuarios operacionales, arquitectos y responsables del negocio.	Escenarios operacionales, entorno de uso, actores, condiciones de operación y flujos de trabajo.

2.1.2a) Relación entre los tipos de documentos

Aunque estos documentos se encuentran estrechamente relacionados, cada uno representa un nivel diferente de refinamiento. El StRS responde a la pregunta sobre *qué necesitan los interesados*; el SyRS establece *qué debe realizar el sistema*; el SRS describe *qué funcionalidades deberá implementar el software*; mientras que el documento OpsCon proporciona el contexto operacional necesario para comprender el comportamiento esperado del sistema dentro de su entorno de operación [1], [6].

2.1.2b) Ejemplo aplicado al proyecto SIGA

En el proyecto SIGA (Sistema Inteligente de Gestión de Aulas), por ejemplo, una necesidad expresada por la Facultad de Ciencias de la Computación y Diseño Digital como "disminuir el tiempo de respuesta ante fallas de infraestructura" formaría parte del StRS. Posteriormente, el SyRS transformaría dicha necesidad en requisitos del sistema relacionados con la detección automática de anomalías mediante sensores IoT. Finalmente, el SRS detallaría requisitos específicos de software, como el requisito funcional RF-08 correspondiente a la generación automática de alertas y el requisito no funcional NFR-01, que establece un tiempo máximo de notificación de sesenta segundos.

2.1.3. Ciclo de vida documental de los requisitos

Los documentos de requisitos evolucionan paralelamente al ciclo de vida del sistema. En consecuencia, no constituyen productos estáticos, sino artefactos sujetos a revisión continua

conforme se refinan las necesidades de los interesados y aumenta el conocimiento sobre el dominio del problema. IEEE/ISO/IEC 29148 establece que los productos de información generados durante la ingeniería de requisitos deben mantenerse sincronizados con las actividades definidas en ISO/IEC/IEEE 15288 e ISO/IEC/IEEE 12207, favoreciendo la consistencia entre los diferentes niveles de especificación [1], [2], [3].

2.1.3a) Etapas de la evolución documental

De manera general, la evolución documental comprende las siguientes etapas:

1. Identificación de necesidades y expectativas de los interesados.
2. Elaboración del documento de requisitos de interesados (StRS).
3. Transformación de necesidades en requisitos de sistema (SyRS).
4. Derivación de requisitos específicos de software (SRS).
5. Validación con las partes interesadas.
6. Establecimiento de líneas base (baselines).
7. Gestión de cambios y control de versiones.
8. Trazabilidad durante implementación, pruebas, operación y mantenimiento.

2.1.3b) Gestión de cambios en proyectos de alta complejidad tecnológica

La correcta administración de estos documentos permite mantener la integridad de la información incluso cuando se producen modificaciones en los requisitos. Esta característica resulta especialmente importante en proyectos con alta complejidad tecnológica, como SIGA, donde la incorporación de nuevos sensores IoT, dispositivos inteligentes o servicios de inteligencia artificial puede originar cambios que deben propagarse de manera controlada hacia los modelos de análisis, diseño, implementación y pruebas.

2.1.4. Buenas prácticas para la documentación de requisitos

Diversos cuerpos de conocimiento coinciden en que la calidad de un proyecto depende en gran medida de la calidad de sus especificaciones. SWEBOK identifica la documentación clara, verificable, consistente y trazable como una práctica esencial para reducir defectos durante el desarrollo del software, mientras que el programa de certificación CPRE de IREB enfatiza la necesidad de emplear una terminología uniforme, estructuras documentales normalizadas y mecanismos explícitos de gestión de cambios [4], [5].

2.1.4a) Principales buenas prácticas

Entre las principales buenas prácticas destacan:

- Utilizar una estructura documental normalizada conforme a IEEE/ISO/IEC 29148.
- Mantener una única fuente de verdad para cada requisito, evitando duplicidades.
- Asignar identificadores únicos a todos los requisitos.

- Mantener la trazabilidad bidireccional entre requisitos, modelos, diseño, código y casos de prueba.
- Diferenciar claramente requisitos funcionales, requisitos de calidad, restricciones y decisiones de diseño.
- Evitar lenguaje ambiguo mediante el uso de verbos verificables, preferentemente utilizando el término “deberá” (*shall*) para expresar requisitos obligatorios.
- Gestionar formalmente el versionamiento y las líneas base de los documentos.
- Validar periódicamente los documentos junto con representantes de todas las partes interesadas.

2.1.4b) Aplicación de estas prácticas en el proyecto SIGA

En el contexto del proyecto SIGA, estas prácticas permitirán que requisitos como el RF-08 (Generación de alertas por anomalías) mantengan una relación explícita con los modelos funcionales, el modelo de datos y los modelos de comportamiento que serán desarrollados en las secciones posteriores de este documento. Asimismo, facilitarán la verificación del requisito no funcional NFR-01, que establece un tiempo máximo de generación de alertas de sesenta segundos, mediante criterios objetivos de aceptación y casos de prueba definidos durante la etapa de validación.

2.1.4c) Cierre y transición hacia el SRS

Como consecuencia, la documentación de requisitos deja de ser únicamente un mecanismo de registro para convertirse en el principal medio de coordinación técnica entre todas las disciplinas involucradas en el desarrollo del sistema. Sobre esta base se construye el Documento de Especificación de Requisitos de Software (SRS), cuyo contenido y atributos de calidad se analizan en la siguiente subsección, conforme a la estructura propuesta por IEEE/ISO/IEC 29148:2018 y su relación con el histórico estándar IEEE 830.

2.2. Documento ERS/SRS según IEEE/ISO/IEC 29148:2018

2.2.1. ¿Qué es el documento SRS/ERS?

La Especificación de Requisitos de Software (Software Requirements Specification, SRS), denominada también Especificación de Requisitos de Software (ERS) en diversos contextos hispanohablantes, constituye el principal producto documental de la Ingeniería de Requisitos para describir de forma precisa, verificable y trazable el comportamiento esperado del software. De acuerdo con IEEE/ISO/IEC 29148:2018, el SRS forma parte del conjunto de productos de información generados durante los procesos de ingeniería de requisitos y sirve como referencia para las actividades de diseño, implementación, verificación, validación, operación y mantenimiento del software [1], [3].

A diferencia de documentos orientados a las necesidades del negocio, como el Stakeholder Requirements Specification (StRS), el SRS documenta los requisitos específicos que deberán implementarse en el software, manteniendo la trazabilidad con los requisitos del sistema (SyRS) y con las necesidades originales de los interesados. Esta separación favorece la gestión de cambios y reduce inconsistencias entre los diferentes niveles de especificación durante el ciclo de vida del sistema [1], [2], [6].

2.2.2. Estructura general del documento SRS

2.2.2a) Contenidos mínimos considerados

IEEE/ISO/IEC 29148:2018 propone una estructura de referencia para la elaboración de documentos de especificación de requisitos, cuya finalidad es facilitar la organización uniforme de la información y promover la trazabilidad entre requisitos, modelos y productos del ciclo de vida. La norma proporciona una plantilla adaptable al contexto del proyecto, permitiendo incorporar únicamente las secciones pertinentes según el tipo de sistema desarrollado [1]. Entre los grupos de contenido habitualmente considerados en un SRS se encuentran:

- Información introductoria y propósito del documento.
- Alcance y contexto del producto de software.
- Definiciones, acrónimos y referencias normativas.
- Descripción general del sistema y su entorno operacional.
- Requisitos funcionales.
- Requisitos de interfaces.
- Requisitos de desempeño y calidad.
- Restricciones de diseño e implementación.
- Requisitos relacionados con datos, seguridad, mantenimiento y operación.
- Información complementaria y anexos cuando corresponda.

2.2.2b) Flexibilidad de la plantilla normativa

Más que imponer un formato rígido, la norma enfatiza que el contenido del documento debe responder a las características del proyecto y satisfacer las necesidades de comunicación entre todas las partes interesadas [1], [4].

2.2.3. Comparación con IEEE Std 830:1998

2.2.3a) Diferencias principales

Durante varios años, IEEE Std 830:1998 constituyó la referencia más utilizada para la elaboración de especificaciones de requisitos de software. No obstante, la evolución de la Ingeniería de Requisitos y la integración de los procesos de ingeniería de sistemas motivaron su sustitución por IEEE/ISO/IEC 29148, la cual amplía significativamente el alcance del estándar anterior e incorpora una visión alineada con los procesos definidos en ISO/IEC/IEEE 15288 e ISO/IEC/IEEE 12207 [1], [2].

Cuadro 2: Comparación entre IEEE Std 830:1998 e IEEE/ISO/IEC 29148:2018

Aspecto	IEEE Std 830:1998	IEEE/ISO/IEC 29148:2018
Enfoque	Software	Ingeniería de requisitos para sistemas y software
Integración con el ciclo de vida	Limitada	Integrada con ISO/IEC/IEEE 15288 e ISO/IEC/IEEE 12207
Tipos de documentos	Principalmente SRS	StRS, SyRS, SRS, OpsCon y otros productos de información
Trazabilidad	Recomendada	Elemento fundamental durante todo el ciclo de vida
Adaptabilidad	Plantilla relativamente fija	Plantillas adaptables según el contexto del proyecto

2.2.3b) Alcance de la actualización normativa

En consecuencia, IEEE/ISO/IEC 29148 no reemplaza únicamente una plantilla documental, sino que incorpora una visión integral de la ingeniería de requisitos, donde la especificación constituye un producto de información conectado con los procesos de gestión, arquitectura, verificación, validación y mantenimiento del sistema [1], [6].

2.2.4. Atributos de calidad de un SRS

La utilidad de una especificación depende tanto de su contenido como de su calidad. IEEE/ISO/IEC 29148 define características deseables para los requisitos individuales, mientras que la familia ISO/IEC 25000 (SQuaRE), particularmente ISO/IEC 25010, proporciona el marco para especificar y evaluar los requisitos de calidad del producto software [1], [7].

2.2.4a) Atributos considerados

En el contexto de un SRS, destacan los siguientes atributos:

- **Compleitud:** el documento debe cubrir todos los requisitos relevantes para el sistema.
- **Consistencia:** no deben existir contradicciones entre requisitos ni entre los distintos modelos asociados.
- **Verificabilidad:** cada requisito debe poder comprobarse mediante inspección, análisis, demostración o prueba.
- **Trazabilidad:** cada requisito debe mantener relaciones explícitas con su origen, diseño, implementación y casos de prueba.

- **Ausencia de ambigüedad:** cada requisito debe admitir una única interpretación por parte de los involucrados.
- **Modificabilidad:** la estructura documental debe facilitar la incorporación de cambios sin introducir inconsistencias.

2.2.4b) Beneficios de estos atributos

Estos atributos permiten incrementar la mantenibilidad del documento y reducir el riesgo de interpretaciones erróneas durante las fases posteriores del desarrollo [4], [7].

2.2.5. Ejemplo aplicado al proyecto SIGA

La diferencia entre un requisito ambiguo y uno correctamente especificado puede ilustrarse mediante el requisito funcional RF-08 del proyecto SIGA.

2.2.5a) Ejemplo inadecuado

El sistema deberá enviar rápidamente alertas cuando exista algún problema en un aula.

El término “rápidamente” carece de un criterio objetivo de aceptación, por lo que el requisito no resulta verificable.

2.2.5b) Ejemplo conforme a las buenas prácticas del SRS

RF-08. El sistema deberá generar una alerta cuando cualquier SensorIoT detecte valores fuera de los umbrales configurados y enviarla al Servicio de Notificaciones en un tiempo máximo de 60 segundos desde la detección del evento.

Este requisito puede relacionarse directamente con el requisito no funcional NFR-01 (eficiencia de desempeño), cuya verificación consistirá en medir el tiempo transcurrido entre la detección de la anomalía y la recepción de la notificación por parte del usuario autorizado. Además, mantiene una relación explícita con la entidad *Alerta*, con el caso de uso CU-04 “Generar alertas por anomalías” y con el modelo de comportamiento correspondiente al ciclo de vida de una alerta, lo que facilita la trazabilidad entre las distintas perspectivas del sistema [1], [7], [8].

En consecuencia, un SRS correctamente elaborado no solo documenta los requisitos del software, sino que también constituye el eje central de la trazabilidad del proyecto, conectando las necesidades de los interesados con los modelos de análisis, los componentes de diseño, las pruebas y los criterios de aceptación. Esta relación será desarrollada en las siguientes subsecciones mediante los modelos de datos, funcionales y de comportamiento aplicados al sistema SIGA.

2.3. Modelos de datos (Perspectiva de datos)

2.3.1. Concepto y finalidad de los modelos de datos

Los modelos de datos constituyen uno de los principales mecanismos de representación utilizados durante la Ingeniería de Requisitos para describir la estructura lógica de la información que deberá administrar un sistema de software. Su finalidad consiste en

identificar los objetos relevantes del dominio, sus atributos, las relaciones existentes entre ellos y las restricciones que regulan su comportamiento, proporcionando una visión estática del sistema independiente de las decisiones de implementación. Esta representación facilita la comprensión del problema, favorece la comunicación entre las partes interesadas y constituye un insumo esencial para las actividades posteriores de diseño, desarrollo y verificación del software [1], [9].

Desde la perspectiva establecida por IEEE/ISO/IEC 29148:2018, los modelos de datos forman parte de los artefactos utilizados para complementar la especificación textual de requisitos, permitiendo representar de forma gráfica información cuya comprensión resultaría más compleja mediante lenguaje natural. Estos modelos contribuyen además a la verificabilidad, consistencia y trazabilidad de los requisitos definidos en el Documento de Especificación de Requisitos de Software (SRS), especialmente cuando el sistema administra grandes volúmenes de información relacionada entre sí [1], [4].

En proyectos de ingeniería de software de mediana y gran escala, los modelos de datos permiten identificar tempranamente inconsistencias, redundancias y omisiones dentro de los requisitos funcionales, reduciendo significativamente el riesgo de modificaciones costosas durante las etapas posteriores del ciclo de vida. En consecuencia, su elaboración constituye una actividad fundamental dentro del análisis del dominio del problema y de la construcción del modelo conceptual del sistema [2], [6].

2.3.2. Diagramas Entidad–Relación

2.3.2a) Concepto y preguntas fundamentales del modelo ER

Uno de los modelos conceptuales más ampliamente utilizados para representar la estructura de la información es el Diagrama Entidad–Relación (Entity Relationship Diagram, ERD). De acuerdo con Maciaszek, este modelo permite describir los elementos principales del dominio mediante entidades, atributos y relaciones, independientemente del sistema gestor de bases de datos o de la tecnología utilizada para implementar la solución [9].

Una **entidad** representa un objeto, concepto o elemento relevante sobre el cual el sistema debe almacenar información. Las entidades poseen un conjunto de **atributos** que describen sus propiedades, mientras que las **relaciones** representan asociaciones existentes entre dos o más entidades del dominio.

Desde una perspectiva conceptual, el modelo ER responde principalmente a tres interrogantes fundamentales:

- ¿Qué información necesita almacenar el sistema?
- ¿Qué características posee cada elemento del dominio?
- ¿Cómo se relacionan dichos elementos entre sí?

La correcta identificación de estos componentes permite construir un modelo consistente del dominio del problema antes de abordar decisiones relacionadas con bases de datos, arquitectura de software o tecnologías de implementación.

2.3.2b) Aplicación al dominio del proyecto SIGA

En el caso del proyecto SIGA, el dominio está conformado por aulas inteligentes equipadas con sensores IoT, dispositivos electrónicos y servicios de monitoreo continuo. Como consecuencia, el modelo conceptual debe representar tanto la infraestructura física

como la información generada por los dispositivos inteligentes y los procesos administrativos derivados de su operación.

2.3.3. Componentes fundamentales del modelo ER

Según Maciaszek, un modelo Entidad–Relación está constituido principalmente por los siguientes elementos [9]:

2.3.3a) Entidad

Corresponde a un objeto distinguible dentro del dominio del problema. Una entidad posee identidad propia y puede ser identificada mediante uno o varios atributos clave.

Dentro del sistema SIGA pueden identificarse, entre otras, las siguientes entidades:

- Aula
- SensorIoT
- GatewayIoT
- LecturaSensor
- Equipo
- Proyector
- Climatizador
- Usuario
- Rol
- Alerta
- SolicitudMantenimiento
- ConsumoEnergetico
- HorarioAcademico
- BitacoraAccion

Cada una representa un concepto independiente cuya información deberá mantenerse durante la operación del sistema.

2.3.3b) Atributos

Los atributos corresponden a las propiedades que describen una entidad. Dependiendo de la naturaleza del dominio, pueden clasificarse como:

- Identificadores (clave primaria).
- Descriptivos.
- Derivados.

- Multivalorados.
- Opcionales.

Por ejemplo, la entidad *SensorIoT* puede contener atributos como:

- idSensor
- tipoSensor
- fabricante
- direccionIP
- estado
- fechaInstalacion

Mientras que la entidad *Alerta* podría incluir:

- idAlerta
- tipo
- prioridad
- descripcion
- fechaHoraGeneracion
- estadoActual

La adecuada identificación de atributos facilita posteriormente la definición del diccionario de datos y la especificación de restricciones de integridad.

2.3.3c) Relaciones

Las relaciones representan asociaciones existentes entre entidades.

Cada relación posee una cardinalidad que indica el número de ocurrencias permitidas entre las entidades participantes.

Las cardinalidades más frecuentes son:

- Uno a uno (1:1)
- Uno a muchos (1:N)
- Muchos a muchos (N:M)

En SIGA predominan relaciones de tipo uno a muchos debido a la naturaleza de los dispositivos IoT. Por ejemplo, una única aula puede contener múltiples sensores ambientales, mientras que cada sensor pertenece únicamente a un aula determinada.

Esta representación facilita la identificación de dependencias entre los diferentes elementos del dominio y constituye la base para la construcción posterior del esquema lógico de datos.

2.3.4. Importancia del modelo de datos en la Ingeniería de Requisitos

Aunque frecuentemente se asocia el modelo ER con el diseño de bases de datos, en Ingeniería de Requisitos su función principal consiste en representar el conocimiento del dominio. IEEE/ISO/IEC 29148 establece que los modelos gráficos complementan la especificación textual de requisitos, permitiendo reducir ambigüedades y mejorar la comunicación entre clientes, analistas y desarrolladores [1].

Asimismo, SWEBOK destaca que los modelos conceptuales favorecen la validación temprana de requisitos al proporcionar una representación visual fácilmente interpretable por las partes interesadas, incluso cuando estas no poseen conocimientos técnicos avanzados [4].

En el proyecto SIGA, por ejemplo, resulta considerablemente más sencillo validar la relación existente entre las entidades *Aula*, *SensorIoT*, *LecturaSensor* y *Alerta* mediante un modelo conceptual que únicamente mediante una descripción textual de los requisitos funcionales.

De igual forma, la representación gráfica facilita identificar posibles inconsistencias, tales como entidades redundantes, relaciones innecesarias o atributos insuficientemente definidos antes del inicio del diseño físico de la base de datos.

2.3.5. Relación del modelo de datos con la trazabilidad de requisitos

Una de las principales contribuciones del modelo ER dentro de la Ingeniería de Requisitos consiste en facilitar la trazabilidad entre las diferentes perspectivas del sistema. IEEE/ISO/IEC 29148 establece que los requisitos deben mantener relaciones explícitas con los modelos utilizados durante el análisis, permitiendo verificar posteriormente que cada necesidad identificada por los interesados ha sido adecuadamente representada dentro de la solución propuesta [1].

En el proyecto SIGA esta trazabilidad puede observarse claramente mediante el requisito funcional RF-08, correspondiente a la generación automática de alertas por anomalías. Dicho requisito se refleja directamente en la entidad *Alerta*, la cual almacena la información generada por los sensores IoT cuando detectan condiciones fuera de los umbrales establecidos. Posteriormente, esta misma entidad será utilizada por el Diagrama de Flujo de Datos para representar el procesamiento de eventos y por el Statechart para modelar el ciclo de vida de una alerta, garantizando así la consistencia entre las perspectivas de datos, funcional y de comportamiento.

En consecuencia, el modelo Entidad-Relación no constituye únicamente una representación de estructuras de información, sino un mecanismo fundamental para asegurar la integridad conceptual del sistema y mantener la correspondencia entre requisitos, modelos de análisis y componentes de software durante todo el ciclo de vida del proyecto. En la siguiente subsección se presenta un modelo conceptual aplicado al sistema SIGA, construido a partir de las entidades identificadas durante el análisis del dominio y representado mediante un Diagrama Entidad-Relación.

2.3.6. Modelo Entidad-Relación aplicado al sistema SIGA

Con base en el análisis del dominio realizado durante la Ingeniería de Requisitos, se identificaron las principales entidades involucradas en el funcionamiento del Sistema Inteligente de Gestión de Aulas (SIGA). Estas entidades representan los objetos relevantes que intervienen en el monitoreo de infraestructura educativa, el procesamiento de informa-

ción proveniente de dispositivos IoT y la gestión administrativa derivada de los eventos detectados.

El modelo conceptual propuesto organiza la información del dominio mediante entidades, atributos y relaciones, proporcionando una visión estática del sistema independiente de cualquier tecnología de implementación. Asimismo, constituye la base para la construcción del diccionario de datos y del modelo lógico que serán utilizados durante las etapas posteriores del desarrollo del software [1], [9].

Las entidades fueron seleccionadas considerando los requisitos funcionales del proyecto, particularmente aquellos relacionados con el monitoreo ambiental, la generación automática de alertas, el control de equipos tecnológicos y la gestión de solicitudes de mantenimiento. En consecuencia, el modelo garantiza la trazabilidad entre los requisitos identificados y la estructura de información requerida por el sistema.

Bibliografía

- [1] *ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering*, Requirements Engineering Standard, Geneva, Switzerland: International Organization for Standardization (ISO) et. al., 2018. dirección: <https://www.iso.org/standard/72089.html>.
- [2] *ISO/IEC/IEEE 15288:2023 Systems and software engineering — System life cycle processes*, System Life Cycle Processes, Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), Institute of Electrical y Electronics Engineers (IEEE), 2023. dirección: <https://www.iso.org/standard/81702.html>.
- [3] *ISO/IEC/IEEE 12207:2017 Systems and software engineering — Software life cycle processes*, Software Life Cycle Processes, Geneva, Switzerland: International Organization for Standardization (ISO), International Electrotechnical Commission (IEC), Institute of Electrical y Electronics Engineers (IEEE), 2017. dirección: <https://www.iso.org/standard/63712.html>.
- [4] *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide) Version 4.0*, 4.^a ed. Los Alamitos, California, USA: IEEE Computer Society, 2024. dirección: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>.
- [5] *Certified Professional for Requirements Engineering (CPRE) Foundation Level Syllabus*, 3.1, Foundation Level Syllabus, International Requirements Engineering Board (IREB), Nuremberg, Germany, 2025. dirección: <https://cpre.ireb.org>.
- [6] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*, 2.^a ed. Cham, Switzerland: Springer, 2025, Part IV: Requirements Artefacts, ISBN: 978-3-662-69204-2. DOI: 10.1007/978-3-662-69205-9. dirección: <https://link.springer.com/book/9783662692042>.
- [7] *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*, SQuaRE Product Quality Model, Geneva, Switzerland: International Organization for Standardization (ISO) e International Electrotechnical Commission (IEC), 2023. dirección: <https://webstore.iec.ch/en/publication/90024>.
- [8] E. Hull, K. Jackson y J. Dick, *Requirements Engineering*, 3.^a ed. London: Springer, 2011, ISBN: 978-1-84996-404-3. DOI: 10.1007/978-1-84996-405-0. dirección: <https://link.springer.com/book/10.1007/978-1-84996-405-0>.
- [9] L. A. Maciaszek, *Requirements Analysis and System Design: Developing Information Systems with UML*, inglés, 1.^a ed. Harlow, England: Addison-Wesley, 2007, ISBN: 978-0-321-44036-5.